

AURORA: Agentic UAV Refinement through Optimization, Reflection, and Assurance

Chi Zhang

Abstract

UAV controller tuning is an iterative experiment: every Candidate must remain bound to its scenario, budget, constraints, Runtime, and retained evidence. **AURORA—Agentic UAV Refinement through Optimization, Reflection, and Assurance**—is the evidence-gated Harness inside DroneDream 1.0. Foundation models interpret intent and rank eligible methods, while deterministic contracts authorize execution, evidence admission, holdout isolation, and final selection. The frozen Evidence 2.8 / Prompt 1.7 online routing campaign qualifies 23/24 responses from 24 calls to the gpt-4.1-2025-04-14 snapshot; its single failing case is retained, and archived prompt histories are not treated as causal comparisons. Separate contract evidence exercises cross-Job memory and fail-closed provenance. A pinned PX4/Gazebo campaign records 6/6 passing x500 Trials across two seeds and nominal, steady-wind, and static-obstacle scenarios, with RMSE 0.332254–0.444452 m and track coverage 0.949113–0.981456. Wind read-back and obstacle creation are retained for both affected scenarios. These results establish the exercised contracts, routes, and injections—not optimizer superiority, broad simulator quality, obstacle avoidance, real-aircraft safety, simulation-to-reality transfer, or statistical generalization. Claims bind to frozen commits and hashes; Windows Rust was not run.

 github.com/ChiZhang-805/DroneDream

1. Introduction

UAV controller tuning is not a single-number search. PX4 layers rate, attitude, velocity, and position controllers, and useful gains shift with propulsion, mass, assembly, and scenario conditions. Its official workflow therefore remains iterative: apply parameters, excite the system, inspect the response, and revise. Software-in-the-loop changes the risk and cost of that loop, not its combinatorics. A credible campaign must choose a method and finite budget, preserve scenario identity, evaluate objectives and constraints, distinguish parameter failure from infrastructure fault, and reserve unseen scenarios for final selection. Derivative-free and Bayesian methods systematize search, from reusable services to trust-region and sparse high-dimensional variants [2, 3, 4, 5, 6, 7, 8], but still assume a correctly formed experiment. They neither translate engineering intent into an auditable program nor prove that its declared conditions were executed as declared in its persisted manifest or reconciled to the retained evidence ledger.

Foundation models provide a useful interface layer, but not experimental authority. Tool-using agent systems show that model reasoning becomes more effective when coupled to explicit tools, machine-readable state, feedback, reflective memory, and bounded execution interfaces [16, 17, 28, 29, 30]. In control optimization, however, a plausible explanation cannot prove that a scenario was applied, a metric came from the intended trial, or a final candidate remained isolated from holdout leakage. Those facts must come from deterministic software and retained evidence. AURORA (Agentic UAV Refinement through Optimization, Reflection, and Assurance), the evidence-gated Harness inside DroneDream 1.0, therefore uses model intelligence to interpret the task and rank specialized methods, while deterministic compilers, versioned contracts, simulator receipts, outcome taxonomy, and a final freeze gate authorize every consequential transition. The model may propose the next action; it cannot declare that action executed or promote an unverified observation. AURORA therefore compiles conversation into a bounded optimization program whose claims are earned by retained evidence. This separation is operational: a route becomes admissible only after the same Candidate, scenario, budget, Runtime, and receipt identity survive validation. If any identity changes, the observation remains unavailable instead of entering memory, selection, or a reported comparison.

This report makes three contributions. First, it specifies the system boundary that separates model reasoning, tool eligibility, simulation execution, and evidence authority. Second, it reports reproducible routing, guard, fallback, and synthetic integration evidence without relabeling software contracts as physical performance. Third, it defines the experiments still required before broader simulator-quality claims are justified. The presentation follows the causal narrative common to modern technical reports in agentic and embodied AI [15, 18, 19], while keeping DroneDream’s claims tied to its own frozen artifacts and independently auditable release boundary for independent regeneration, external verification, and reuse without querying mutable application state or hidden Runtime state.

1.1 Product scope

The product boundary is simulation-first. The desktop application guides scenario design, experiment planning, execution, evidence review, recovery, and reporting. The public website explains and distributes the product, hosts documentation and community features, and exposes subscription entry points. Supabase supplies identity, community features, quota accounting, and access to the managed-model gateway. DroneDream 1.0 keeps PX4 and Gazebo simulation local to the owned Runtime. The boundary is equally explicit about what is not claimed: this report does not certify real-aircraft safety, guarantee simulation-to-reality transfer, or authorize unbounded autonomous flight actions. A model may propose or route work, but it may not convert an unverified result into a completed tuning outcome. AURORA must earn that transition with evidence under the frozen release contract.

2. Related work and design rationale

Black-box optimization supplies the search machinery, but not the experimental constitution. CMA-ES provides a robust derivative-free evolutionary strategy, while Vizier and BoTorch formalize reusable optimization services and modern Bayesian-optimization primitives. TuRBO addresses local search in high dimensions, and SAASBO models sparse axis-aligned structure [4, 5, 6, 7, 8]. Efficient Global Optimization established the surrogate-and-acquisition loop for expensive black-box functions, practical Bayesian optimization showed how to carry that loop into noisy model selection, and the field review makes the remaining choices around priors, acquisition, constraints, and parallelism explicit [31, 32, 33]. These methods motivate DroneDream’s portfolio, yet each remains conditional on a correctly specified objective, feasible domain, trial budget, and trustworthy observation stream. DroneDream’s distinct engineering problem begins one level above the optimizer: it must decide which method is eligible, prove which scenario was actually applied, and prevent infrastructure faults or holdout outcomes from entering the learning history or contaminating any later comparison after release.

Robotic foundation models demonstrate broad semantic and action priors, but they solve a different authority problem. RT-1 and RT-2 established large-scale vision-language-action learning; Open X-Embodiment and OpenVLA expanded cross-embodiment data and open model access; π_0 and $\pi_{0.5}$ explored flow-based action generation and heterogeneous co-training; JoyAI-RA continued this line with a multi-source autonomy foundation model [9, 10, 11, 12, 13, 14, 15, 20, 21, 27]. DroneDream does not claim an end-to-end flight policy or place a language model in the control loop. It uses model reasoning only to interpret intent and rank eligible tools; deterministic software retains authority over experiment construction, simulator execution, evidence admission, final selection, release, and rollback through every audited product transition and later replay.

Data scale and benchmark breadth have advanced faster than evidence semantics. Open X-Embodiment, BridgeData V2, DROID, and AgiBot World expand the diversity of robot experience; RoboCasa and RoboTwin 2.0 scale simulation tasks and controlled perturbations [11, 22, 23, 24, 25, 26]. These resources strengthen policy training and comparative evaluation, but their success metrics do not answer whether a local UAV tuning run applied the intended scenario, whether a failed trial was physical or infrastructural, or whether holdout evidence leaked back into search. DroneDream therefore treats dataset coverage, optimizer quality, and evidence validity as independent axes: benchmark breadth or policy scores cannot substitute for evidence about the exact experiment that generated a reported result.

Tool-using agents clarify why that separation matters. SWE-agent shows the value of an explicit agent-computer interface, and AlphaEvolve couples model proposals to evaluators and an evolutionary loop [16, 17, 28, 29, 30]. ReAct interleaves reasoning with environmental actions; Toolformer studies learned tool invocation; Reflexion uses feedback as episodic memory. Together they locate capability in interfaces, evaluators, and feedback loops. DroneDream adopts that interface principle but never treats free-form reflection as experimental evidence. Only structured, provenance-bound, policy-eligible outcomes may update decision memory; failure taxonomy, holdout isolation, and the final freeze gate prevent generated narratives from replacing the versioned simulator receipts and outcome records that support a claim.

3. System and method

The architecture assigns a single authority to every transition. The desktop owns user intent and interaction; the FastAPI backend and worker validate job transitions and persist bounded state; the AURORA compiles context, filters tool eligibility, and admits only trusted outcomes; the owned Runtime supplies pinned PX4/Gazebo dependencies; and cloud services provide identity, community data, quota accounting, and a managed-model gateway without exposing the platform key to clients. Separating these roles prevents a router choice from becoming an executable experiment until deterministic compilers, validators, and Runtime probes agree on the same versioned contract before launch or retry. The authority graph is reconstructible from persisted identities: every transition names its input state, approving contract, execution owner, and evidence output. Retry and recovery must preserve those identities, so neither path can silently convert a proposal into an accepted execution or an unavailable observation into a successful, admissible outcome after retry or during later replay and publication review.

3.1 AURORA: the evidence-gated Harness

AURORA is a constrained decision system around simulation. Context binds parameters, scenarios, history, budgets, and failures. Capability eligibility removes incompatible tools, while a phase gate exposes only methods suited to exploration, recovery, refinement, diversification, or verification. The router ranks that executable surface and the optimizer proposes Candidates. Verification forces every reduced-fidelity strategy to fidelity 1.0 before acceptance. Provenance-bound memory admits trusted outcomes, while the final gate freezes holdout results and rejects incomplete artifacts. Each stage narrows authority, so fluent output cannot bypass an executable contract or test. The retained decision trace records the selected tool together with every filtered alternative and its rejection basis during later replay without changing the frozen source identity or claim scope in audit review.

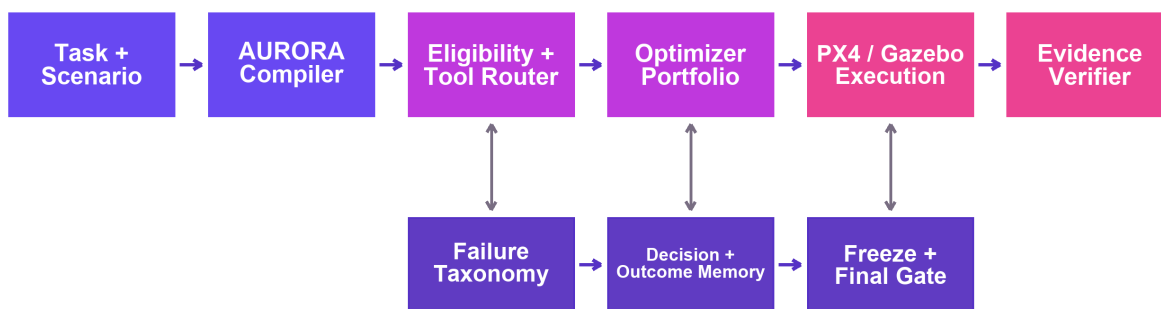


Figure 1. AURORA's evidence-gated optimization loop inside DroneDream 1.0. Provenance-complete, policy-eligible outcomes alone may influence routing memory or pass the final selection gate.

- Desktop boundary — entry appears only after every required launch check passes. Terminal UI state alone authorizes readiness and exit; active work or unsaved drafts trigger an attributable recovery path instead of silent process termination. The same readiness record supports later recovery, support, audit replay, and service use.
- Backend boundary — drafts, Jobs, APIs, and artifacts cross only through validated transitions with stable failure codes. A rejection retains its reason, source identity, and replay receipt; it cannot become persisted, dispatched, or published work. The receipt remains independently replayable before any later release decision at scale.
- AURORA authority — only provenance-complete, policy-eligible evidence enters context, routing, reflection, or memory. Holdout, infrastructure-fault, legacy, incomplete, and drifted records remain unavailable to routing, selection, and replay. No later narrative can reinterpret a rejected row as admissible learning evidence.
- Runtime boundary — a physical claim must bind executable effect, expanded source, pinned dependency commits, the accepted customer Runtime, a published ledger, and an independently checked receipt. A launched process without that chain cannot support public release or customer-facing release qualification or review.
- Cloud boundary — authentication, community data, quota ledger, and model gateway remain server-authoritative; provider secrets never enter desktop logs or artifacts, and each managed-model debit must retain an authenticated account, quota-ledger entry, provider disposition, and reconcilable service receipt under its frozen billing contract.

Claim-time inputs are immutable before launch. Schema `dronedream.trial-execution-attempt-claim/v3` and digest `execution_input_sha256` bind Trial, Candidate, and Job inputs in an immutable snapshot. Outcome Contract 2.20 authorizes the scenario matrix and constructs **TrialContext** without rereading mutable rows. Checks classify changed inputs as **INPUT_EVIDENCE_DRIFT**, coordinated rewrites as **OUTCOME_CONTRACT_DRIFT**, and unauthorized fields as **SCENARIO_CONTRACT_DRIFT**. Completion locks Job before Trial; historical receipts remain verifiable, new claims use v3, and the original projection stays available for audit replay under the same claim boundary across concurrent execution, replay, export, and release review or customer acceptance.

Physical claims are concurrency-safe and fairly scheduled. Execution Claim Gate 1.1 combines PostgreSQL FOR UPDATE SKIP LOCKED, a conditional lease update, bounded retries, and an attempt-count fence. Concurrent workers converge on a unique physical execution and accepted outcome for each logical Trial, with no duplicate evidence across the exercised pool. Scheduling Gate 1.0 chooses the least-served eligible Job and preserves FIFO within it. A paired-Job regression alternates sequential claims, preventing desktop-lane monopoly; hosted priority and resource-cost scheduling remain outside the current local scheduling contract, audit replay, and evidence envelope for hosted work without weakening per-tenant fairness or ownership isolation at scale.

Model feedback is case-separated. Prompt Schema 2.3 assigns provider-safe training `case_N` aliases in frozen order, keeping same-type cases with different weights or numeric configurations distinct. Each admitted case carries its weight, seed count, metrics, and closed failure counts; raw IDs, unsupported text, and holdout evidence stay outside the prompt. This removes feedback-grouping ambiguity without expanding model authority or claiming simulator fidelity, and preserves canonical order when matched Trials arrive in reverse. The separation is preserved in every provider request and retained routing receipt for later independent inspection and audit replay when cases are recomputed or revisited after release under the same frozen prompt schema and case order.

Reproducibility extends below the random seed. A non-unique eigenvector factor can rotate an identical seeded normal sample when another BLAS kernel resolves repeated covariance eigenvalues differently. DroneDream instead uses the unique symmetric positive-definite covariance square root and canonical unit-vector spelling before discrete projection. The frozen optimizer transcript hashes executed candidates, losses, feasibility, route, fidelity, and cohort position, while non-causal GP/CMA diagnostic matrices remain outside that decision hash [1]. This boundary lets supported machines replay identical candidate sequences across machines before any provider decision receives credit. The transcript is compared at the executed-Candidate boundary rather than against optimizer diagnostics that may vary by library implementation. A replay mismatch therefore identifies a decision-relevant divergence before a provider route or reported observation is accepted, keeping numerical implementation drift separate from evidence drift during independent replay and release review on a different supported machine and BLAS stack.

Feedback is verified again at the model boundary. Optimizers, the closed-tool router, and direct proposer share a training-feedback compiler. It reconstructs canonical Trial rows and checks the content-addressed outcome against Candidate identity, generation, parameters, and Trial evidence before exposing a score. Sibling fields cannot override those values. Divergence is quarantined rather than turned into a bad-parameter penalty; migration-era rows remain explicitly unsealed and ineligible for learning, routing memory, or final selection [1]. The rule preserves auditability across both current and migration-era experiment histories and their later replay without retroactive reinterpretation or an unverifiable score entering routing or model-mediated recovery across release boundaries.

Reflection is compiled from verified outcomes rather than free-form model self-report. Evidence 2.8 retains the bounded **observed_outcome** cohort summary and adds a distinct cross-Job memory gate. Only a terminal source Job whose decision and Candidate cohort are **verified_complete** can materialize an experience; retrieval requires the same authenticated user, a different source Job, and the exact structural task family. A 90-day TTL, per-Job and global revocation, deletion cascade, parameter-catalog binding, contract-version binding, and source-receipt binding fail closed. Prompt 1.7 may receive at most six anonymous closed-structure observations, excluding owner or source IDs, timestamps, parameters, seeds, holdout values, credentials, and error text. Decision Trace 1.4 preserves the selected surface and labels every such observation **observational_not_causal**. Incomplete, legacy, revoked, expired, or drifted cohorts remain **unavailable**; a zero-dispatch decision remains not applicable. The frozen evaluation passes 10/10 fixtures—two positive retrievals and eight isolation or lifecycle negatives—with zero provider, network, or simulator calls and zero provider-identifier leaks. Memory may guide a later decision, but it cannot assign optimizer benefit, provider benefit, safety improvement, or portfolio-child credit without a separately controlled intervention.

- **COMPLETED** is admitted as successful optimizer evidence only when a usable metric and modern receipt bind the exact Candidate, Trial, scenario, objective direction, and execution provenance; a terminal label without that complete chain remains unavailable for optimizer learning, final selection, export, or any later performance statement in release.

- FAILED with a trusted domain failure code is admitted as a non-success because it may identify an infeasible or unsafe simulated parameter region without being mislabeled as infrastructure failure; the failed attempt stays in the optimization denominator with its scenario and Candidate identity intact through every later aggregate and arm comparison.
- Infrastructure faults, cancellation, malformed evidence, and conflicting terminal states are quarantined and cannot update search or routing memory under any later decision path or replay; each rejected row retains a stable quarantine reason and remains visible to operational diagnostics and independent evidence accounting without later relabeling.
- Holdout-only outcomes remain sealed for final evaluation, never return to training history or provider-visible reflection, and become selectable only through the frozen final gate under the predeclared metric and admissibility policy. That separation preserves an untouched comparator for the registered final decision or release.

Infrastructure failures, cancellation, malformed or internally conflicting evidence, and holdout-only outcomes are isolated from optimizer learning and routing memory. Admission requires a terminal state, an allowed domain failure code where applicable, a content-addressed Candidate-to-Trial match, and complete provenance for the executed scenario. Rejected rows retain a stable quarantine reason for diagnosis but cannot update scores, route preferences, recovery memory, or the next candidate proposal. This evidence boundary prevents a simulator crash, stale callback, or evaluation-only result from being learned as if it described the quality of a parameter set. The retained experiment ledger makes every quarantine decision independently inspectable without allowing the rejected observation to affect later search.

4. Experiments and evidence

This section keeps unlike evidence units separate. Development and policy-holdout corpora test tool eligibility; source ablations test named guards; the fallback campaign tests recovery equivalence under a fixed mock budget; and synthetic integration tests orchestration on a declared non-physical objective. Every result is recomputed from persisted records and paired with a claim boundary. PX4/Gazebo assertions remain isolated until the signed customer Runtime passes a provenance-bound acceptance campaign, preventing software checks and proxy outcomes from being pooled into one headline score or silently promoted beyond their declared evidence class in any publication-facing claim during external review, customer acceptance, or independent replay of the frozen release.

4.1 Current and archived development-routing evidence

The current online artifact binds Evidence 2.8, Prompt 1.7, Tool 2.1, the gpt-4.1-2025-04-14 model snapshot, JSON-schema responses, temperature 0, top-*p* 1, seed 20260728, and the same 24-case development corpus. Its 24 provider calls completed in 77.8 seconds. The frozen response is independently regraded at 23/24 (95.83%), with a 37.5 percentage points advantage over the 14/24 best constant policy. It qualifies against minimums of 75% overall, 15 percentage points over the best constant, and 2/3 in every category; tight budget is the lowest category at exactly 2/3. The seed does not make provider output deterministic, and qualification applies only to this exact frozen call set, snapshot, controls, corpus, and scoring rule used for that qualification decision and no broader release.

The sole failure, `tight_budget_expensive_matrix`, selected TuRBO where the accepted set was multi-fidelity MOBO or the optimizer portfolio. The archived provider artifact remains separately regradable: the first Prompt 1.5 freeze scored 19/24 and was unqualified; a second freeze under the same Prompt 1.5 template scored 21/24 and qualified. Prompt 1.6 then recorded 24/24 acceptable selections. Evidence v9 marks that Evidence 2.7 / Prompt 1.6 row **contract_current=false**. These archived calls identify neither causal prompt lift, lower simulator loss, nor generalization across unseen tasks, vehicles, scenarios, or provider outputs after release or deployment.

Table 1: Current and archived routing accuracy

Comparator	Acceptable selections	Evidence treatment
Current Prompt 1.7 online freeze	23/24	Qualified exact-contract campaign
Prompt 1.6 archived freeze	24/24	Historical development qualification
Prompt 1.5 first freeze	19/24	Unqualified
Prompt 1.5 repeat freeze	21/24	Qualified historical run
Shared Prompt 1.5 misses	3 cases	Local progress, mixed history, and tight budget
Best constant policy	14/24	Deterministic corpus baseline
Uniform random expectation	5.625/24	Analytical expectation

The current 23/24 current-contract qualification retains its one tight-budget miss instead of normalizing it away. The first Prompt 1.5 freeze missed five cases; the 21/24 repeat retained three of those errors, and the archived Prompt 1.6 freeze selected acceptable tools for all 24 cases. Because these provider calls were not paired across prompt versions, the sequence localizes observed routing disagreements but cannot attribute their changes to prompt wording. A controlled paired study remains necessary before any revision receives causal credit or publication.

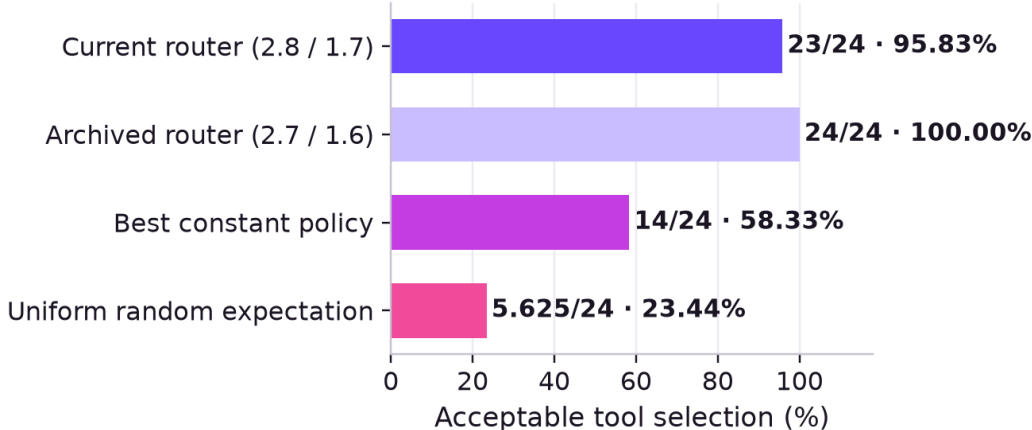


Figure 2. Acceptable selection rates for the current Evidence 2.8 / Prompt 1.7 online freeze, the archived Evidence 2.7 / Prompt 1.6 freeze, and corpus baselines. The unpaired results are not a causal prompt comparison, optimizer outcome, or simulator result.

4.2 Deterministic routing-policy holdout

A separate 16-case corpus freezes exact eligible-tool sets at capability boundaries and binds the corpus, case IDs, compiled policy inputs, and development-corpus hash. It passes 16/16 exact comparisons. The runner performs zero model calls, simulator runs, or feedback writebacks, and a central flow guard rejects use of these results as prompt examples, training data, development evidence, or runtime feedback. Cases exercise dimension, constraint, multi-objective, fidelity, feasible-history, and restart boundaries so the exact-set assertion detects both an incorrectly admitted tool and an incorrectly excluded one. Because labels become visible when the repository is published, this artifact verifies deterministic policy behavior but is not described as a permanently blind benchmark. The zero-activity record is part of the claim: it proves the holdout evaluator did not call a provider, launch a simulator, or write feedback while resolving labels. A future change that introduces any of those side effects would invalidate comparison with this frozen artifact and require a new protocol version.

Table 2: Deterministic routing-policy holdout

Measure	Result	Claim boundary
Eligible-tool sets	16/16 exact matches	Current production eligibility only
External activity	0 calls; 0 simulator runs	No model or outcome evidence
Adaptive writeback	0; flow guard rejects	Evaluation artifact only
Label visibility	Published with repository	Not permanently blind; labels now public

4.3 AURORA source-contract ablations

The ablation suite asks a narrow question: whether each named source guard changes an otherwise identical deterministic decision at the boundary it is designed to protect. For every guard, the production contract and one intentionally weakened comparator receive the same fixed probes, expected result, and scoring rule. Production satisfies 25/25 probes, whereas the matched weakened variants satisfy 6/25 after their designated protection is removed. This construction demonstrates enforceable behavior for fallback, provider trust, outcome isolation, and tool eligibility; it is not a statistical effect estimate and does not measure optimizer quality, simulated control performance, flight safety, or readiness. The comparison is diagnostic because each weakened arm changes one named admission rule while inputs and expected outputs stay fixed. A failed probe therefore localizes source-boundary enforcement; it estimates neither customer frequency nor optimizer benefit in physical controller campaigns or release decisions across independently retained seed blocks and later acceptance reviews.

Table 3: AURORA source-contract ablations

Guard	Protected boundary	Cases	Production	Weakened
deterministic fallback	Provider-independent recovery path	4	4/4	1/4
provider trust filter	Trusted provider evidence enters decisions	5	5/5	2/5
scenario/outcome isolation	Holdout and fault evidence stay outside learning	6	6/6	2/6
scenario profile context	Profile evidence enters every decision	5	5/5	0/5
tool eligibility gate	Incompatible tools remain blocked	5	5/5	1/5

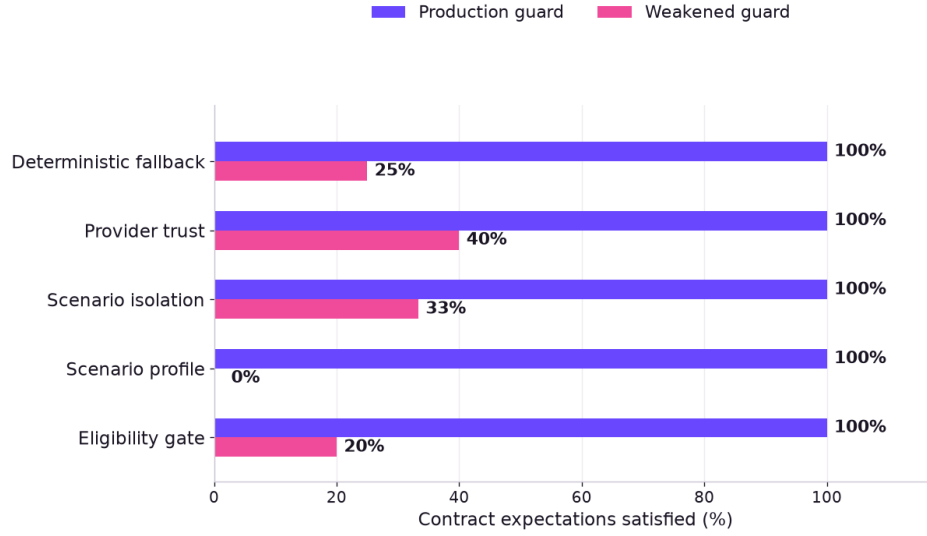


Figure 3. Constructed deterministic software-contract probes for each AURORA guard. The weakened comparison measures fail-closed boundary enforcement, not simulator performance or causal effect.

4.4 Fail-closed outcome equivalence

Five fixed seed blocks compare three matched-budget arms: direct portfolio, provider-error fallback, and invalid-response fallback. The campaign runs 15 Jobs and persists 579 Trials under the frozen budget.

Both fallback paths match the direct arm in all 10 blockwise comparisons across Candidates, Trials, budget, winner, holdout loss, failure count, and evidence completeness. All 15 arms reach 100% evidence completeness; a runtime socket guard observes zero network connection attempts and no real credential is used. By freezing the task, seed, design, budget, mock adapter, terminal accounting, and equivalence fields, the protocol isolates the recovery route as its only declared difference from the direct comparator. Equality also binds attempt ordinals, Candidate ancestry, terminal dispositions, and receipt paths, so a recovery cannot replace the failed observation or silently create a second eligible outcome. Any discrepancy remains a failed equivalence check rather than being normalized into a shared terminal status or aggregate. The receipt records the mismatched field and preserves the affected seed block for replay.

Table 4: Fail-closed outcome equivalence

Arm	Job runs	Persisted Trials	Outcome vs direct
Direct deterministic portfolio	5	193	Reference arm
Provider-error fallback	5	193	5/5 exact
Invalid-response fallback	5	193	5/5 exact

The campaign establishes deterministic fallback equivalence on `MockSimulatorAdapter` under the frozen task, seeds, budget, adapter, and comparison fields. Equality covers persisted Candidates and Trials, selected winner, holdout loss, failure count, evidence completeness, and absence of network use; it does not imply that every future provider error is recoverable. Because all three arms execute the same deterministic portfolio after routing, the result does not show LLM superiority, causal AURORA benefit, PX4/Gazebo performance, controller quality, or flight safety. That bounded conclusion is replayable from retained Candidates, Trials, receipts, and comparison fields under the frozen protocol, using the archived JSON and CSV exports as the independent audit surface. A future recovery claim must rerun the matched design with an actual provider failure class and retain any arm-specific divergence instead of assuming that mock equality generalizes. This is the minimum evidence needed to distinguish recovery coverage from recovery performance. The matching fields are deliberately stricter than terminal status alone: Candidate inventory, Trial denominator, winner identity, loss, failures, and receipt completeness must agree within each seed block. A shared COMPLETED label without that identity-level equality would fail the frozen comparison.

4.5 Tool-use behavior

Eligibility is computed before routing. CMA-ES and the deterministic portfolio remain general fallbacks. Constrained multi-objective BO is admitted only for constrained or multi-objective tasks; multi-fidelity BO requires reducible replicates; TuRBO and surrogate CMA-ES require scored feasible history; and SAASBO or BIPOP CMA-ES require their respective high-dimensional or restart preconditions. A missing precondition removes the tool rather than asking the model to reason around it. The current 24-case distribution therefore describes routing coverage under Evidence 2.8 and Prompt 1.7, not the relative optimization quality of those methods or a causal improvement over earlier prompt freezes [4, 6, 7, 8]. Its frozen selections are portfolio 9, TuRBO 5, constrained MOBO 3, BIPOP-CMA-ES 3, surrogate CMA-ES 2, SAASBO 1, and multi-fidelity MOBO 1. These are route frequencies, not tool-quality estimates; every response is checked against the phase-qualified manifest before dispatch, and no method-specific superiority follows without physical outcomes. Figure 4 is therefore a coverage diagnostic for the validated router surface, not an empirical ranking or outcome comparison among the seven selected method families; it does not measure how any method changes controller behavior under a shared scenario, seed, and execution budget, and it cannot establish repeatability beyond the frozen routing corpus itself.

Evidence 2.9 closes an offline budget-and-accounting contract, not an online planning-quality claim. Three fixed seed blocks compare a direct portfolio arm with a scripted multi-tool arm under equal ceilings of two generations and 40 Trials. All 3/3 blocks preserve configured budget parity across six arm runs. The scripted arm records six verified generations, including three multi-tool generations, and accounts for 12/12 decision calls across plan and revision stages. The `MockSimulator` run makes zero provider and network calls and uses no real credential. Its first repository-root launch failed with `ModuleNotFoundError(app)` before starting a Job or contacting a provider, network, or simulator; the failure and the source-bound CLI fix remain in the receipt. This closure verifies closed-schema plan history, multi-tool dispatch, revision, provenance, and cost accounting. It does not measure real LLM planning quality, optimizer superiority, physical fidelity, real-flight performance, or causal AURORA benefit. No result authorizes a deployment qualification in any customer release decision.

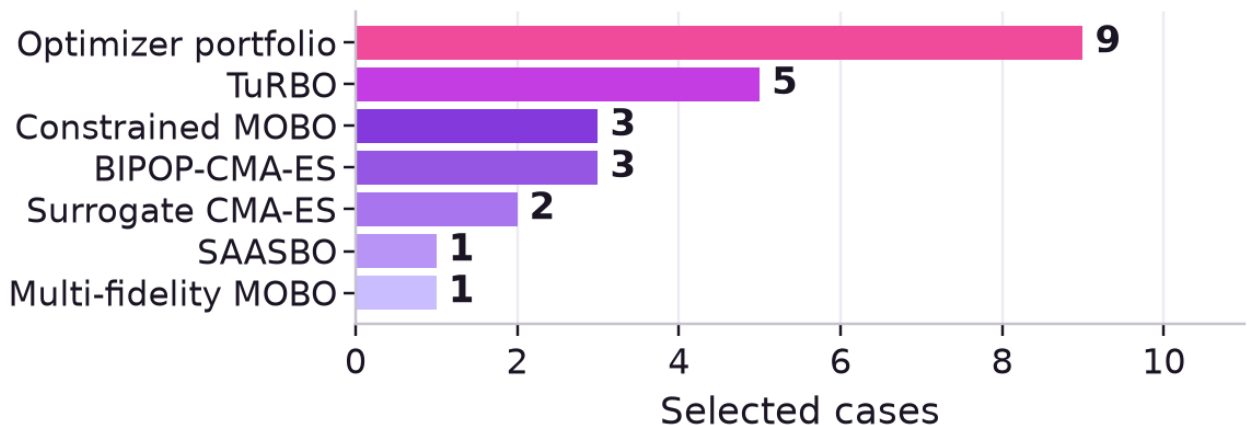


Figure 4. Tool distribution across the current 24-case Evidence 2.8 / Prompt 1.7 online freeze; counts describe selection frequency, not optimizer quality, causal prompt improvement, or provider determinism.

Evidence 2.8 and Evidence 2.9 share the phase-qualified registry but not an evaluation unit. Figure 4 counts online tool selections for 24 routing cases; the offline closure measures plan-history and accounting behavior for six scripted/mock arm runs. Neither denominator may be substituted for the other, and neither licenses an optimizer-outcome or physical performance comparison under any deployment or customer setting.

4.6 Synthetic integration campaign

This verified proxy experiment is deliberately labeled non-physical. It exercises compilation, candidate generation, deterministic evaluation, persistence, selection, holdout accounting, export, and evidence recomputation across ten scenario schemas. The local adapter uses frozen seeds and coefficients; no PX4, Gazebo, vehicle dynamics, network provider, or customer credential is involved. The result tests orchestration and evidence flow only; it does not establish controller quality, simulator fidelity, physical performance, deployment safety, or simulation-to-reality transfer.

Simulation-queue behavior is also an experimental-validity concern. Execution Claim Gate 1.1 binds each authorized Trial to a content-addressed snapshot, while Scheduling Gate 1.0 admits the least-served eligible Job's next FIFO Trial. Regressions cover concurrent claimants, physical-attempt accounting, retry ownership, and alternating claims across paired Jobs. These checks do not improve the proxy objective; they prevent reported budgets and cohort comparisons from being distorted by duplicate execution, starvation, or shared-Runtime queue monopolization. The ledger retains every authorized claim and terminal disposition for independent replay after every failure, retry, recovery, and ownership transition. Queue correctness is assessed on the admitted denominator, not worker throughput alone. Every retry keeps its original Job, snapshot, and physical-attempt identity, so concurrency cannot create a second eligible observation or remove the failed attempt that preceded recovery from the retained denominator, evidence replay, or later aggregate value. Replay is evaluated as a state invariant rather than a throughput target. After every claim, the Job's next-Trial cursor, attempt ordinal, Candidate snapshot, and terminal disposition must agree across database and worker views. A retry may re-enter execution only from its recorded failure state; it cannot acquire a new budget, seed, scenario, or Candidate identity. The receipt also binds the prior failure to the recovered attempt, so a worker restart cannot replace one authorized observation with another or drop an unavailable cell. These invariants expose concurrency faults before any recovered value enters selection and preserve every declared arm, scenario, and seed in later campaign tables; throughput remains separately reported and cannot relax claim eligibility, denominator preservation, or final evidence admission during replay, release review, or customer qualification under an independently accepted Runtime and frozen release receipt.

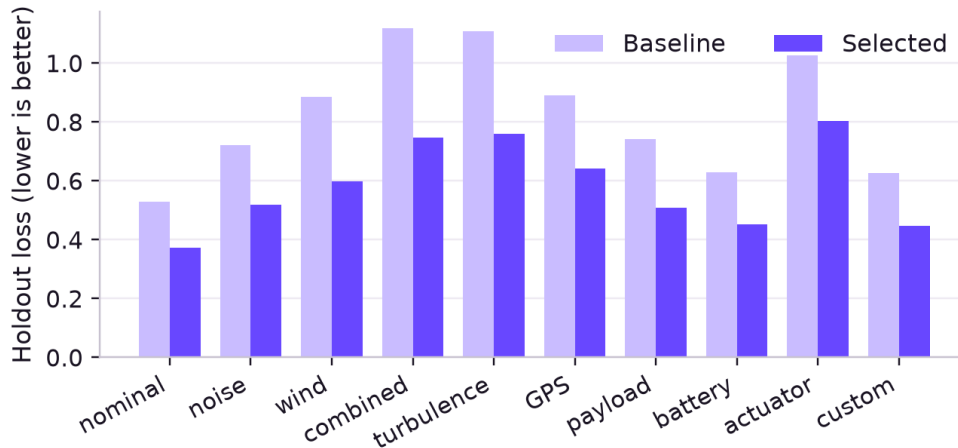


Figure 5. Baseline and selected holdout loss across ten frozen synthetic scenarios; evidence metadata records a mock simulator, false physical fidelity, and no provider network access.

Across the frozen 61-candidate mock campaign, aggregate holdout loss fell from 0.82811 to 0.58525 (29.327% relative). All ten named scenarios improved; scenario-wise relative reductions ranged from 21.8% to 33.2%. The selected candidate matched the exhaustive mock oracle in every scenario under the frozen proxy objective, candidate grid, and deterministic tie-break rule. The exporter recomputes the aggregate and per-scenario values from all persisted Trials and confirms that the selected Candidate belongs to the evaluated set. These measurements verify the closed synthetic campaign only; they do not predict the size or direction of improvement in a physical simulator under any flight condition without matched physical execution and retained evidence from every matched seed.

The artifact declares `physical_fidelity=false`, `simulator_backend=mock`, and `provider_network_access=false`. Its scenario scores are generated by a frozen deterministic proxy objective rather than flight dynamics, sensors, actuator models, or environmental plugins. The claim is limited to orchestration, artifact completeness, and optimizer selection within that proxy. It is not PX4/Gazebo evidence, real-aircraft evidence, sim-to-real evidence, a safety claim, or a basis for physical deployment decisions; any later physical claim therefore requires independent, provenance-bound PX4/Gazebo measurements under the frozen release protocol with matched seeds and retained failures and independent recomputation under a signed, frozen release protocol and receipt set.

4.7 Scenario-wise proxy behavior

Scenario-wise reporting exposes heterogeneity that a single aggregate loss would conceal. The heat strip and the quantitative table use the same ten frozen mock scenarios, selected Candidate, deterministic objective, and persisted holdout records as the integration campaign above. Values are paired within scenario: baseline and selected losses are recomputed from the ledger before the relative reduction is calculated. The resulting range shows that every declared proxy scenario improved under this closed campaign, while preserving the same non-physical boundary. These rows do not estimate uncertainty across vehicles or simulators and cannot be interpreted as PX4/Gazebo, real-aircraft, safety, or sim-to-real evidence under any vehicle, Runtime, or customer setting.

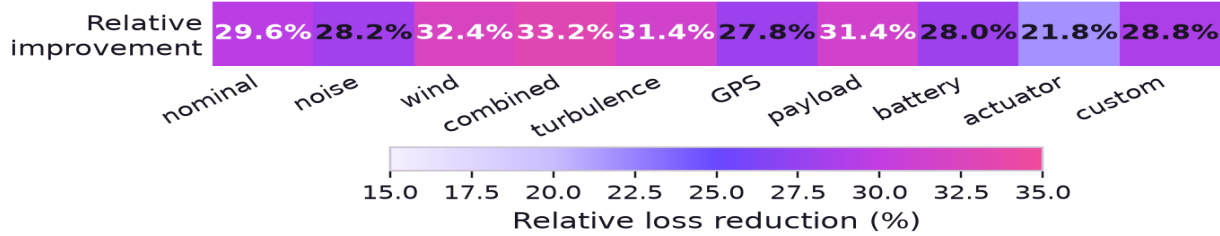


Figure 6. Relative loss reduction by mock scenario.

The strip should be read as a paired descriptive view of the frozen proxy campaign, not as a confidence interval or cross-vehicle generalization result. Each cell recomputes the relative change from the baseline and selected loss stored for the same named scenario, so a row cannot borrow evidence from a different seed, scenario, or Candidate. The narrow range is useful because it exposes whether one scenario dominates the aggregate, yet it remains a property of this deterministic mock objective. Publication-grade claims still require independent seeds, physical simulator receipts, retained failures, and uncertainty computed from the frozen PX4/Gazebo campaign rather than synthetic proxy assumptions; the strip cannot establish variance, robustness, or safety beyond that proxy.

Table 5: Scenario-wise synthetic holdout results

Scenario	Baseline	Selected	Reduction
nominal	0.5299	0.3733	29.55%
noise perturbed	0.7221	0.5185	28.20%
wind perturbed	0.8863	0.5992	32.39%
combined perturbed	1.1193	0.7474	33.23%
turbulence	1.1093	0.7609	31.41%
gps dropout	0.8899	0.6422	27.83%
payload changed	0.7429	0.5093	31.44%
battery degraded	0.6288	0.4530	27.96%
actuator delay	1.0262	0.8024	21.81%
custom	0.6264	0.4463	28.75%

4.8 Physical scenario evidence boundary

The frozen dedicated Runtime campaign uses PX4 6ea3539157ca358c70a515878b77077af7d4611d, Gazebo Sim 8.14.0, gz-harmonic 1.0.0-1~noble, MAVSDK 3.15.3, pyulog 1.2.3, and Ubuntu 24.04. Its fixed 2-seed $\times$ 3-scenario $\times$ 500 matrix records 6/6 passing circle-track Trials: RMSE 0.332254–0.444452 m, maximum error 1.496354–1.498899 m, and coverage 0.949113–0.981456. Both steady-wind Trials bind /world/default/wind_info ENU (0, 2, 0) m/s read-back; both static-obstacle Trials bind /world/default/create=true and SDF SHA-256 prefix 9824260767a952b1. The obstacle sits outside the nominal track to exercise entity injection, not collision avoidance. The signed installer Runtime and advanced effects remain unaccepted.

- Execution boundary — the dedicated Runtime records 6/6 successful, passing Trials for exactly two seeds and three $\times$ 500 scenarios; this verifies the retained matrix, not a wider reliability distribution, optimizer advantage, released customer-performance claim, or cross-scenario reliability estimate across independently repeated installations and audits.
- Injection boundary — both wind runs retain ENU (0, 2, 0) m/s read-back, and both obstacle runs retain successful create replies plus the same SDF hash; obstacle placement outside the circle track tests injection only and supplies no collision-avoidance result or collision-response evidence under alternative obstacle placements or trajectories or winds.

- Failure boundary — four chronological probes retain invalid input enum, dubious PX4 ownership, unsupported sensor noise, and NumPy scalar parsing. Attempt 4 is a post-fix ULog diagnostic only; it is excluded from the six-Trial numerator and cannot erase retained failures in any later customer-acceptance artifact or audit.
- Qualification boundary — customer admission must repeat the campaign in the signed Runtime and pin vehicle/world assets, controller schema, scenarios, seeds, budgets, baseline arms, and failure taxonomy in one versioned receipt. Acceptance also requires independent outcome recomputation on a separately provisioned installation.
- Denominator rule — retain every timeout and failed Trial; a missing receipt stays unavailable rather than being interpreted as a safe or successful run, and aggregate rates must preserve that disposition across every arm, scenario, seed block, and independently recomputed publication table without dropping unavailable runs from the denominator or seed.

5. Evaluation protocol for publication-grade results

The main benchmark will compare methods under the same pinned Runtime, controller catalog, starting design, train/holdout split, trial budget, failure thresholds, and blocked randomization schedule. Final-test manifests must be frozen before any arm observes their outcomes. The benchmark ledger must bind the Runtime image, simulator and controller commits, scenario files, parameter domains, seed blocks, initial design, stopping rule, failure-code mapping, and metric implementation. Each arm receives equal access to training outcomes and identical terminal accounting, while the final-test partition remains write-protected. Reported aggregates must be independently recomputed from persisted Trials with uncertainty, effect size, and failures retained in the denominator. The same ledger must retain every excluded run and the exact reason its result remained ineligible during external recomputation and final publication stage, before any public comparison or customer-acceptance claim.

The controlled comparison should progress from a default policy through fixed CMA-ES, a preregistered specialist, the deterministic portfolio, a memory-disabled AURORA arm, and the complete AURORA system.

Every arm must share the same scenario set, initialization, trial budget, and terminal accounting. Specialist selection must be frozen before final-test outcomes are visible, portfolio comparisons require a matched exploration budget, and the memory ablation must retain identical routing inputs. This sequence isolates increasingly specific system contributions without treating a method-name difference as causal evidence or attributing it to AURORA without a preregistered intervention, matched denominator, and independently recomputed evidence set.

- Primary outcomes bind frozen holdout loss, feasible-campaign rate, baseline improvement, time-to-target, and terminal failure rate to every reported arm and seed block, with paired uncertainty and effect size computed from the retained scenario-level comparison units. These units govern preregistered acceptance and arm-to-arm inference at release.
- Secondary outcomes: wall time, model tokens and cost, router latency, recovery success, scenario-wise regret, and artifact completeness for every frozen comparison arm and seed block, reported alongside termination cause and evidence eligibility so efficiency cannot hide a higher failure burden or incomplete audit surface at publication time or review.
- Use at least five independent seeds for engineering diagnostics and preferably ten for report-level comparisons across every reported arm; freeze seed blocks before final execution and publish paired scenario-level uncertainty before making any superiority or repeatability claim across the preregistered final-test matrix and all baseline arms in scope.
- Keep timeouts, crashes, infeasible terminations, and failed Trials in the denominator under their predeclared terminal categories; never impute a successful loss, discard a difficult seed, or replace a missing receipt with an optimistic outcome, and never reclassify an unavailable execution after completed-arm results or rankings become visible during audit.
- Report paired uncertainty, effect sizes, sample counts, and failure dispositions for each arm and scenario family; never publish superiority from one seed, one track, a post-hoc metric, or an unregistered subset of completed runs. The registered denominator and subset must remain visible in every release table and appendix ledger.

The registered comparison matrix is therefore the authoritative denominator. A release must enumerate planned and realized cells before opening final outcomes, preserve unavailable cells with their terminal reason, and regenerate every aggregate from that matrix. A summary that cannot be traced back to the frozen arm, seed, scenario, and failure registry remains exploratory and cannot enter customer qualification or any released comparative study. Its receipt must name the registry digest and the exact planned and realized cell inventory for audit.

6. Product reliability and operations

The desktop readiness gate is part of the product’s technical contract. The progress bar may report completion only after every check that DroneDream can perform at launch has reached a terminal pass state. The entry action must not become visible while a required check is pending or failed. After entry, runtime failures still need explicit diagnostics because network loss, disk damage, permission changes, and external service changes cannot be permanently prevented. The gate therefore certifies a timestamped launch snapshot rather than promising an error-free future session, and post-entry faults must remain recoverable, attributable, and visible without silently invalidating prior work for every affected run. Its receipt should preserve probe timestamps, dependency versions, the service endpoint, account disposition, and the first failing stage. Recovery verification should replay install, upgrade, repair, offline, and damaged-cache paths without allowing stale success to satisfy a fresh launch or conceal the newly failing dependency under the same receipted probe sequence. The receipt must also retain the exact recovery action and the first post-recovery probe so a green launcher state remains attributable to one tested environment rather than becoming a generic assertion of platform health after repair or upgrade.

Readiness is a conjunction, not a progress animation. The launcher requires a fresh supported-system probe, one owned and compatible Runtime, a healthy local service, no mandatory update, and an account accepted by the local API. Missing, stale, mismatched, unreachable, or still-checking states remain blocking. Only the current gate may mark readiness complete, change the environment badge to CHECKED, and reveal the entry action. Quantitative release evidence still needs time-to-ready p50/p95, false-ready rate, and a clean Windows/upgrade matrix across fresh install, upgrade, repair, and duplicate-download paths before any release is accepted for public distribution on a supported Windows architecture or the entry action is exposed across the declared release matrix.

Exit and account operations use the same recoverability principle. Closing warns about an active simulation or unsaved draft, stops work that cannot survive process termination, and preserves a recoverable draft whenever possible without trapping the user in the application. Managed-model usage consumes the included quota through a server-side ledger before BYOK is required; provider receipts must later be reconciled against that ledger. Remaining evidence therefore concerns draft-recovery and exit success rates plus quota-receipt consistency, not a new product capability claim, across every managed-provider transaction, recovery path, and audit replay retained by the service ledger, with a reconciliation receipt preserved for later independent audit of account state and quota events.

7. Limitations and next experiments

AURORA’s present evidence establishes enforceable software boundaries, reproducible mock-campaign behavior, and one dedicated 6-Trial PX4/Gazebo execution matrix. It does not establish broad optimization superiority or accepted customer-Runtime physical-simulator performance. The distinctions below are active claim constraints, not generic future-work disclaimers: each identifies a missing evidence unit, uncontrolled confounder, or unaccepted environment. Keeping them explicit prevents development routing accuracy, deterministic guard probes, synthetic proxy gains, and the two-seed physical matrix from being pooled into a stronger conclusion than their separate protocols support. The next experiments therefore prioritize frozen matched budgets, independent recomputation, signed Runtime acceptance, and scenario-level uncertainty over time under independently rebuilt deployments with retained failure histories and audit replay before any result advances toward customer acceptance.

- No result in this report establishes real-aircraft safety, certification, or direct simulation-to-reality transfer; those claims require separately governed hardware evidence, operational risk controls, and accepted flight testing outside the present software-contract scope and do not inherit assurance from the mock, WSL, or policy-only artifacts reported here.
- The current Evidence 2.8 / Prompt 1.7 online freeze qualifies at 23/24 for one model snapshot and 24 calls; it establishes neither broad model generalization nor optimizer, simulator, flight-performance, or safety gain.
- Evidence 2.9 exercises equal-budget scripted multi-tool planning and accounting over three seed blocks, six arm runs, and 12/12 accounted calls on a mock backend with zero provider or network calls. It cannot establish real LLM planning quality, optimizer superiority, physical fidelity, real flight, or causal Harness benefit under any release.
- Archived Prompt 1.5 and Prompt 1.6 freezes record 19/24, 21/24, and 24/24. The unpaired histories establish neither causal prompt lift nor deterministic provider behavior under the reported seed and controls after release.
- The 10/10 cross-Job memory fixtures exercise retrieval, isolation, and lifecycle gates without measuring online provider effect, optimization gain, physical fidelity, real-aircraft performance, safety, or certification in review.
- The current 16-case policy holdout verifies deterministic eligibility only; its published labels are not permanently blind and require a future concealed benchmark with sealed labels, preregistered scoring, independent custody, and a frozen separation between development examples and final evaluation cases under external final-test custody and audit.

- The source-contract ablation demonstrates named guard behavior, not causal optimization-quality lift; each weakened comparator changes a software protection under constructed probes and therefore cannot estimate controller performance, simulator fidelity, or benefit under real execution noise across matched scenario and seed blocks with uncertainty.
- The frozen four-arm mock ablation changes protocol behavior in 15/15 full-arm comparisons, but its 5/5 receipt-only-memory isolations remain inconclusive; the scripted policy does not authorize a causal performance claim outside a physical, matched-budget, multi-seed intervention with independent outcome recomputation and retained failures.
- The 10/10 fallback result verifies deterministic recovery on a frozen mock protocol, not that AURORA improves objective value beyond recovery equivalence or supports causal attribution; all arms converge on the same deterministic portfolio, so the retained comparison isolates dispatch recovery rather than optimization quality or causal lift.
- The ten-scenario campaign is synthetic mock evidence used to exercise orchestration and evidence flow; it is not physical scenario coverage, vehicle-dynamics validation, disturbance fidelity, or a substitute for signed-Runtime PX4/Gazebo outcomes. It cannot qualify vehicle dynamics, disturbance realism, or control quality in the signed Runtime.
- The dedicated Runtime matrix has only two seeds per nominal, wind, and static-obstacle scenario. The obstacle is outside the nominal track, so creation proves injection rather than avoidance; signed customer Runtime repetition and wider seed blocks remain necessary before reliability, customer acceptance, or statistical generalization across installations.
- Optimizer comparison, physical component ablation, confidence intervals, cost/latency measurement, and UX telemetry remain pending. Each requires matched budgets, retained failures, frozen manifests, and independently recomputed outputs under its declared evidence class for external release review and recomputation at scale.
- External baselines such as expert manual tuning or PX4 AutoTune must be included only when controller, vehicle, track, budget, metrics, and failure accounting are genuinely aligned; unmatched access to resets, prior knowledge, simulator time, or terminal exclusions must be reported as a protocol difference rather than hidden inside a pooled ranking.

7.1 Threats to validity

The report's evidence units answer different questions and cannot be pooled into one score. Routing cases measure tool eligibility, source probes exercise named guards, fallback arms measure recovery equivalence, and synthetic scenarios measure a deterministic proxy. Substituting one unit for another remains a construct-validity error even when every recorded value is correct and the units remain explicitly separate in every release.

The same implementation sometimes generates, executes, and verifies an artifact. A comparative campaign therefore still requires independent recomputation, immutable manifests, negative controls, and failure injection before any optimization-quality difference is described as causal. The physical receipt remains first-party frozen evidence rather than third-party validation, audit, or certification under the present campaign or any later deployment decision.

Statistical validity requires the scenario, not an optimizer Trial, to remain the primary comparison unit. Trials in one trajectory share initialization, history, stopping conditions, and candidate ancestry, so treating them as independent understates uncertainty. A publication result must freeze seed blocks, report paired scenario-wise differences, retain terminal failures in the denominator, and present effect sizes with intervals rather than one aggregate percentage. Hyperparameters, method choice, and stopping rules must be fixed before the final partition opens; exploratory findings remain useful but cannot replace the frozen confirmatory comparison across every arm or its reported causal estimate for any registered arm, scenario family, customer deployment, or release decision under review.

Operational reproducibility requires more than code and a controller vector. A repeatable run must bind the owned Runtime image, PX4/Gazebo commits, vehicle and world assets, controller schema, scenario manifest, optimizer settings, seeds, environment, and evidence verifier. It must retain cancellations, timeouts, rejected callbacks, and post-launch changes rather than successful Trials alone. The ledger already supplies the source commit, test totals, digests, and reproduction command; the remaining gap is a signed customer-Runtime campaign executable on another machine without repository-specific state, hidden assumptions, operator interpretation, or independent auditability during replay across machines, maintainers, and independently provisioned environments. The rerun must reproduce the same planned and realized registry, unavailable cells, and terminal dispositions before any metric is compared across installations, external benchmarks, or customer qualification reviews. The replay receipt must preserve that registry digest and the complete unavailable-cell disposition map for audit.

External validity is bounded by controller, vehicle, world, and disturbance combinations actually executed. Many Trials in one x500 setup do not cover fixed-wing vehicles, payload changes, actuator saturation, estimator faults, or other controller structures. A broader benchmark therefore needs a declared scenario matrix with held-out combinations, not a convenient collection of worlds. Every cell must explain its distinct operating condition, mutable parameters, and invariant feasibility constraints. Results must be aggregated within and across vehicle families so a strong mean cannot conceal a systematic failure in one operationally important subgroup or declared scenario family under final evaluation before a broader cross-platform claim is admitted or published.

Measurement validity requires more than scalar holdout loss: retain trajectory error, settling time, overshoot, control effort, saturation, constraint violations, termination cause, wall-clock cost, energy use, and units.

Each metric needs frozen units, sampling rules, preprocessing, and direction. Feasibility remains a gate because a lower aggregate can hide oscillation or excessive actuator demand within one feasible trajectory.

Primary metrics and admissibility rules must be registered before optimization, while diagnostics remain visible for failure analysis; post-hoc metric choice cannot redefine a failed campaign as successful after execution once final outcomes become visible or a customer-facing ranking has already been produced or released.

The next comparative campaign prioritizes matched-budget PX4/Gazebo optimizer arms, physical component-level AURORA ablations, failure injection, and the readiness/exit matrix. It freezes the Runtime, controller catalog, scenario manifests, larger seed blocks, budgets, eligibility policy, and terminal taxonomy before any final arm runs. Matched baselines include fixed CMA-ES, a preselected specialist, the deterministic portfolio, and isolated AURORA variants. Signed JSON and CSV artifacts must retain failures and preserve the physical and statistical boundaries already exposed by the six-Trial matrix. The ledger must also show which planned comparisons never executed and why in the retained receipt. A causal provider-prompt study must pair Prompt 1.5, Prompt 1.6, and Prompt 1.7 calls on identical cases, model snapshot, call order, budget, and scoring rule; retain every raw response; and report paired category changes with uncertainty instead of selecting the best freeze. That design separates both current and archived routing qualifications from any future causal prompt-lift claim. The causal estimator must use paired case-level contrasts and retain category-specific disagreements, refusals, and parse failures in the denominator. The released audit artifact must preserve every paired case, call order, and response disposition for independent recomputation across every arm without selective exclusion and every published table. For external recomputation, the registry must enumerate the planned and realized seed block for every arm and preserve unavailable cells. A verifier should reproduce each table from that registry without querying application state or choosing a favorable subset after observing outcomes during publication preparation, customer review, or independent audit.

8. Reproducibility and source ledger

Figures 3, 5, and 6 trace to evidence bundle v9, canonical SHA-256 prefix d33c308ce3b47138; Figure 1 is a schematic. Figures 2 and 4 also consume the current online-routing artifact at software head ef00362, file SHA-256 prefix d2359e0540aa. Its manifest file and canonical prefixes are 6b625b8c0a6a and 3ab3830ecf79; implementation source is aeffa01a81. The exact 24 provider calls completed in 77.8 seconds and were independently regraded without rerunning the campaign, contacting the provider again, or mutating evidence.

Evidence 2.9 is separately bound to source 136a1e3 and evidence head 15603c6. Its artifact file and internal SHA-256 prefixes are b20060eb76bf and e5c27308a9be; manifest file and internal prefixes are db78d1c8c999 and 3c49e2b96b3f. The CSV file prefix is 3911632e6dfe, and the retained generation/failure receipt prefix is 611783c9388b. These bytes authorize only the offline equal-budget, scripted/mock plan-history and accounting contract under the frozen mock Runtime; they authorize no online planning-quality or optimizer-outcome claim.

The v9 bundle separately binds c1222c9 to receipt head 8102ffe. Its exact-source logs record 1,204/1,204 tests in 854.71 seconds plus 28/28 focused checks in 6.98 seconds; receipt wall intervals are 864.488 and 8.451 seconds. Ruff and mypy remain unreceipted, and Windows Rust was not run. Cross-Job artifact and manifest canonical prefixes are e1abf46c2aa44e9b and b8d55e358d5642cb; the ten-case evaluation is deterministic in-memory SQLite evidence rather than an online provider or simulator study, physical-performance result, or deployment qualification.

The physical campaign independently binds 86273db to 3b09951. Manifest file and canonical prefixes are 4ac760428ead1506 and 9440bb24f25dbf07; receipt file and internal prefixes are 8f7e1c953338ae87 and 4746dd220583d64b. Its 159-file tree and 598-source-file / 154-retained-file inventories reverify without mismatch. Embedded identifier 5f0f62c789680e5e and handoff identifier c373dc9e4964301a do not resolve as Git commits; the reachable chain is c373dc992d43ff92 to 5f0f62c707c541ae to 3b09951bbea2e5f1. The report therefore verifies frozen bytes and reachable source while retaining the source-binding defect as an explicit provenance limitation for external reviewers, customer acceptance, and downstream release claims during independent replay.

Execution Claim Gate 1.1 and Outcome Contract 2.20 bind each Trial/Candidate/Job snapshot and scenario matrix before evidence admission. Evidence 2.8 / Prompt 1.7 / Decision Trace 1.4 binds the cross-Job memory contract and its **observational_not_causal** projection. The current online freeze qualifies at 23/24 for its exact model snapshot and controls, while the archived Prompt 1.6 freeze records 24/24 and Prompt 1.5 histories remain visible at 19/24 and 21/24. These unpaired observations are not causal prompt evidence. Policy holdout passes 16/16, and the fallback campaign matches the direct arm in 10/10 comparisons across 15 Jobs and 579 Trials. The non-comparative physical matrix records 6/6 passing Trials across two seeds and three scenarios and retains four failure probes. Latest website source 8710553 to receipt-only head 96aba5d records 13/13 focused checks, 325/325 frontend tests, typecheck, lint, application and shared builds, 11/11 deployment tests with 22/22 subtests, and 5/5 performance routes. Its desktop English and Chinese download copy each occupies two lines with final-line fills 0.895 and 0.835; the mobile layout-only audit reports zero violations. The receipt binds release-truth ancestor fd7e770 to 87e049c, which separately records 100/100 browser checks and 118 checksum entries across 119 files. No deployment occurred; TLS, dual-target publication, production Supabase/payment, and two differing unsigned public installers remain external gates. Git-object replay rejects missing bytes, and the two-commit pattern authenticates only bytes present at the source freeze; a receipt never widens the associated claim boundary across later audits, releases, publications, customer communications, acceptance decisions, or independently repeated qualification reviews under the frozen source ledger, and never authorizes an absent claim, substituted artifact, later source, or unrecorded execution [1].

9. Conclusion

DroneDream 1.0 positions AURORA as an evidence-gated Harness for UAV controller optimization in simulation: model reasoning interprets intent and coordinates eligible methods, while deterministic contracts retain authority over inputs, execution, evidence admission, and completion. The frozen release demonstrates exercised software boundaries through routing, holdout-policy, guard-ablation, fallback-equivalence, claim-receipt, case-separated feedback, and a 10/10 cross-Job memory contract evaluation. That memory result is observational and non-causal; it establishes only the enumerated software fixtures. The current 23/24 online routing qualification is likewise bounded to 24 calls, one frozen model snapshot, and the exact development corpus retained for review.

Evidence 2.9 separately demonstrates equal configured budgets and complete plan-history accounting for three scripted/mock seed blocks. Its zero-provider, zero-network closure is not evidence of online LLM planning quality, optimizer benefit, simulator fidelity, or flight performance or customer acceptance at release.

A dedicated Runtime also records a fixed two-seed, three-scenario PX4/Gazebo matrix with 6/6 passing Trials and verified wind and obstacle injection under its fixed campaign, environment, and retained receipts.

Neither evidence class establishes optimizer superiority, broad stability, collision avoidance, signed customer-Runtime acceptance, real-aircraft safety, or simulation-to-reality transfer. The next decisive step is therefore a larger matched-arm campaign plus signed-Runtime repetition and independent recomputation, without weakening AURORA's provenance and leakage boundaries during later evidence campaigns or releases.

The current release therefore changes what future experiments can safely assert before it changes the empirical flight result itself. Claim v3, Outcome Contract 2.20, input/outcome/scenario drift rejection, and cross-Job fair scheduling make concurrent attempt counts, parameter provenance, and scenario eligibility independently auditable. The retained four-probe failure history shows why post-fix diagnostics must remain separate from successful Trials. A decisive external campaign must still freeze PX4/Gazebo versions, manifests, larger seed blocks, budgets, metrics, and baseline arms before execution; retain every timeout and failed Trial; and recompute signed outcomes outside the application process. This separation keeps AURORA's demonstrated software assurances and narrow execution evidence distinct from optimization-quality claims that only a matched comparative study can establish with preregistered uncertainty across every retained arm, failure, and scenario family in the final registry.

Appendix A. Release-contract verification

The matrix separates receipted software checks from product claims. A.1 and A.2 define protected mutation and report-export contracts; Table 6 reports frozen receipts and the recomputed layout contract. Each count is source-bound and measures neither controller quality, physical-simulator fidelity, real-flight safety, nor causal optimization benefit. The map prevents software release verification from being mistaken for physical or optimizer performance; unsupported cells remain unavailable, never normalized into passing engineering evidence or any public customer qualification claim.

Table 6: Release-contract verification matrix

Verification surface	Cases	Passed	Claim boundary
Subject-focused supplement	28	28	Memory/routing/report/evidence paths
Frontend validation receipt	325	325	Tests plus typecheck/lint/build
Report layout contract	Recomputed	Required	Rendered body/list policy
Backend validation receipt	1,204	1,204	Exact-final full run; 28-check focused supplement

A.1 Experiment-report and reflection evidence policy

Downloadable reports project retained evidence: generation ancestry, parameter deltas, metrics, terminal Trial counts, and proposal rationale; missing lineage remains unavailable. The authenticated server snapshot fixes export tier and blocks redirect-based entitlement bypass. Reflection requires a receipt for the exact modern Candidate cohort. Legacy, incomplete, or drifted cohorts stay unavailable, holdout values remain sealed, zero dispatch remains not applicable, and observations never become causal tool credit.

A.2 Frozen report projection

The exporter rebuilds persisted Job, Candidate, Trial, parameter, metric, rationale, holdout, and entitlement evidence rather than interface state. Missing receipts stay unavailable, failures stay in the denominator, and prose cannot invent causes. Free pages receive a native vector seal after pagination; authenticated Plus and Pro pages omit it. Redirects cannot change tier, trace, or stable bytes, and the contract does not qualify controller, simulator, or real-aircraft performance.

A.3 Qualification handoff

A claim advances only when evidence class, schema, source, Runtime, and acceptance command are frozen. Evidence 2.8 / Prompt 1.7 / Trace 1.4 qualifies the enumerated cross-Job software contract, while the current online routing freeze qualifies 23/24 exact-contract selections. The archived Prompt 1.6 freeze records 24/24 and Prompt 1.5 histories at 19/24 and 21/24; none supports a causal provider-effect comparison. The physical matrix qualifies six x500 executions and retained wind/obstacle injections, not optimizer superiority or broad generalization. Customer qualification still needs matched arms, wider seed blocks, signed-Runtime repetition, retained failures, and independent recomputation; flight safety and sim-to-real transfer remain unclaimed.

A.4 Frozen component outcome ablation

The offline ablation locks five seed blocks across four arms. Its 20 Jobs persist 554 Trials with complete evidence, zero network calls, and no real credential. Five of 15 paired comparisons show a protocol difference; ten do not. Five receipt-only-memory isolations remain inconclusive rather than becoming a causal component claim.

Table 7: Frozen four-arm component outcome ablation

Arm	Jobs	Trials	Tool sequence	Finding
Full AURORA	5	120	constrained MOBO to TuRBO	Reference
No decision memory	5	120	constrained MOBO to portfolio	No observed difference
No outcome reflection	5	120	constrained MOBO to portfolio	No observed difference
Fixed portfolio	5	194	portfolio to portfolio	Observed difference
Receipt-only isolation	5 pairs	—	matched route and outcome	Inconclusive

A.5 Phase-qualified execution surface

Eligibility asks whether a tool can honor the Job; phase policy asks whether it should execute now. Evidence 2.8 intersects both before schema, provider manifest, validation, trace, or dispatch. Prompt 1.7 and Decision Trace 1.4 persist the current surface, and the 23/24 online freeze records its exact routing decisions. Archived Evidence 2.7 / Prompt 1.6 rows retain their original labels. Every phase retains deterministic fallback; filtered tools are rejected, leaving an auditable admission trace across every retained phase transition and later replay.

Fidelity is a separate contract. Multi-fidelity methods may screen below full fidelity, but verification, the final generation, and last capacity preserve a fidelity-1.0 path. Acceptance, receipt, scenario, and holdout gates prevent a cheap screen from becoming the terminal winner; the receipt records screening and verification fidelity at terminal selection.

Table 8: Phase-qualified search-role surface

Plan phase	Role count	Primary search intent	Fidelity rule
Exploration	6	Global, sparse, and multi-fidelity discovery	Budget policy
Recovery	4	Restart and deterministic recovery	Budget policy
Refinement	5	Local and surrogate exploitation	Budget policy
Diversification	7	Broad escape from stalled regions	Budget policy
Verification	5	Confirm the strongest eligible Candidate	Fidelity 1.0
Balanced	8	Preserve the complete qualified registry	Budget policy

A.6 Evidence-source ledger

Table 9: Evidence-source and claim-boundary ledger

Evidence surface	Scale	Source ID	Authorized claim
Current online routing	23/24; 24 calls	Evidence 2.8 / Prompt 1.7	Exact-contract development qualification
Provider routing freezes	19/24, 21/24, 24/24	Prompt 1.5 / 1.6	Archived development routing only
Cross-Job memory contract	10/10	Evidence 2.8 / Prompt 1.7	Enumerated software behaviors; non-causal
Multi-tool budget accounting	3 blocks; 6 arms; 12 calls	Evidence 2.9	Offline scripted/mock contract only
Policy holdout	16/16	Policy holdout v3	Eligibility-set determinism
Source-contract ablation	25/25 vs. 6/25	Contract ablation v2	Fail-closed guard behavior
Fallback outcome campaign	15 Jobs, 579 Trials	Fallback campaign v1	Matched offline recovery
Component outcome ablation	20 Jobs, 554 Trials	Component ablation v2	Observed protocol differences
Synthetic scenario campaigns	2 × 61 Candidates	Coverage v3 / gen. v1	Seed and mixed-shift reporting
PX4/Gazebo scenario matrix	6 Trials; 4 probes	Physical campaign v1	Dedicated-Runtime execution only
Website validation chain	325 tests; 5 routes	Typography receipt v1	Local validation only; no deployment

A.7 Report publication gate

The report is released from a dedicated source commit rather than from a mutable working tree. That commit contains the LaTeX source, report-owned figures, evidence-reference manifest, claim ledger, and validation code, but not its own attestation. Software and website evidence remain read-only Git objects addressed by branch, commit, path, and SHA-256, so the report cannot silently replace an upstream artifact while preserving its displayed label.

A clean exact-source checkout builds twice with the source commit time exported as `SOURCE_DATE_EPOCH`. The two PDFs must be byte-identical before publication. The structural audit then rejects unlocated prose, cross-page paragraphs, invalid links, gray text, LaTeX warnings, and overflow. Explanatory paragraphs require at least 80% last-line occupancy; one- to three-line list items require at least 90%. Longer list items are reported separately as explanatory list blocks rather than being compressed into the short-item rule.

Visual review renders every page and checks the left-aligned brand, rounded abstract frame, heading hierarchy, readable figure and table labels, centered placement, unsplit tables, non-overlapping graphics, single-column references, and page-bottom balance. Copy or layout is revised when a gate fails; hidden text, forced last-line justification, and filler captions are prohibited. The receipt-only attestation finally binds the source commit, PDF digest, page images, audits, and manifest digest without creating a self-referential source hash. Downstream website work may consume only the final PDF and its hash, not the report source, scripts, ledger, or upstream evidence bytes.

A.8 Evidence-reference semantics

Every external evidence entry separates the tested subject from the commit that publishes its receipt. The verifier first proves both Git objects exist, then checks ancestry, reads the exact blob at `commit:path`, and recomputes the file SHA-256. JSON artifacts with internal canonical digests are normalized again after removing only their self-digest field. A later software head may therefore retain an older online-routing freeze as an ancestor while publishing a newer, separate Evidence 2.9 artifact; the two protocols remain independently addressable and cannot overwrite one another's claim boundary.

Evidence 2.8 online routing, Evidence 2.9 offline budget accounting, the v9 software bundle, the PX4/Gazebo Runtime campaign, and website validation each keep distinct source IDs and admissible claims. Missing objects, hash drift, non-ancestor provenance, schema drift, or an unexpected current-version substitution fails verification. The report stores only those identifiers and digests. It never vendors the upstream JSON, CSV, logs, Runtime inventory, browser receipt, or retained physical campaign tree into its own source commit.

A.9 Claim and figure regeneration

The claim ledger maps each publication statement to one manifest source and a set of JSON-pointer, CSV, or exact-text assertions. It also rejects superseded totals and labels, so an older passing receipt cannot silently return as a current result. Positive counts are paired with negative boundaries: zero provider access does not become provider quality, scripted/mock execution does not become physical fidelity, and a two-seed Runtime matrix does not become statistical generalization. The audit checks every required body literal only after the immutable source assertions pass.

Report-owned figure scripts apply the same separation. They read frozen Git blobs, rebuild chart inputs deterministically, and compare the new pixels with the checked-in report media. Current routing and archived routing remain labeled independently; v9 software probes remain non-physical; PX4 scenario metrics retain their two-seed denominator. Visual review then checks label size, axis legibility, centering, and overlap at rendered-page scale rather than treating source resolution as proof of readable publication output.

A.10 Failure handling and handoff

An upstream verification failure never authorizes a substitute from the latest branch tip. The report first classifies whether the missing unit is a Git object, a file digest, an internal canonical digest, a source ancestry edge, or a declared schema field. Only the affected claim remains unavailable; unrelated frozen evidence can still be audited under its own source ID. Raw evidence is not regenerated on the report branch, and a source-binding defect stays visible until the producing line publishes a new receipted artifact.

Numeric consistency is checked at the claim level rather than by search-and-replace across prose. Counts, durations, versions, model snapshots, scenario denominators, units, and qualifying thresholds are asserted against the source that authorizes them. A displayed total may appear in the abstract, main evidence section, limitations, conclusion, appendix, and ledger only when those occurrences preserve the same boundary. Historical Prompt 1.5 and 1.6 results, current Prompt 1.7 routing, Evidence 2.9 budget accounting, and PX4 execution therefore remain separately named even when they share one report page.

The final handoff exposes a stable PDF, its SHA-256, and the receipt needed to reproduce the validation decision. The receipt records the source commit, page count, deterministic build result, audit digests, and full visual-review inventory. It does not grant downstream ownership of the LaTeX, chart generators, claim ledger, or evidence manifest, and it does not convert local validation into website deployment, customer Runtime acceptance, independent certification, or flight approval. Any such transition requires its own source-bound receipt and release gate.

A.11 Layout validation semantics

The geometry audit reconstructs source paragraphs from rendered word coordinates and measures the last line against the widest line in the same block. Headings, formulas, code, captions, tables, references, and list markers are classified separately from explanatory prose. Explanatory paragraphs must reach 80%; list items of at most three rendered lines must reach 90%. The rule is satisfied by editing meaning or layout, never by forcing the last line, hiding glyphs, inserting meaningless copy, or enabling `raggedbottom`.

Page-bottom coordinates are a screening signal rather than permission to ignore the rendered page. Keeps for headings, unsplit tables, and non-breaking paragraphs can create legitimate structural movement, so every page is still inspected at publication scale. Tables must remain centered and wholly on one page; figure labels must remain legible and non-overlapping; the bibliography remains single-column and final. Whitespace is resolved through substantive explanation and reflow while preserving the evidence boundary, readable type, and the natural rhythm of the reference layout.

As a final page-bottom diagnostic, the reviewer may temporarily append a normal one-line sentinel after the terminal body block. If the sentinel wraps or moves to the next page, the preceding page is within one baseline of capacity; the sentinel is then removed. If it remains on the same page, the source needs substantive explanation or a legitimate reflow before publication. The sentinel never enters the released PDF, claim ledger, text extraction, or visual-review receipt. It uses the same font, line width, and keep rules as the target page; smaller or hidden tokens are invalid because they understate capacity and bypass visual review.

References

- [1] DroneDream, “Release evidence ledger,” ver. 1.2, evidence bundle v9, current online-routing freeze, and offline multi-tool budget closure, software evidence head 15603c6, Jul. 2026.
- [2] PX4 Development Team, “Multicopter PID tuning guide (manual/basic),” PX4 User Guide, 2026. [Online]. [Accessed: Jul. 27, 2026]. Available: [PX4 Docs](#)
- [3] PX4 Development Team, “Simulation,” PX4 User Guide, 2026. [Online]. [Accessed: Jul. 27, 2026]. Available: [PX4 Docs](#)
- [4] N. Hansen and A. Ostermeier, “Completely derandomized self-adaptation in evolution strategies,” *Evol. Comput.*, vol. 9, no. 2, pp. 159–195, 2001, doi: 10.1162/106365601750190398. [DOI](#)
- [5] D. Golovin et al., “Google Vizier: A service for black-box optimization,” in *Proc. 23rd ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, 2017, pp. 1487–1495, doi: 10.1145/3097983.3098043. [DOI](#)
- [6] M. Balandat et al., “BoTorch: A framework for efficient Monte-Carlo Bayesian optimization,” in *Adv. Neural Inf. Process. Syst.*, vol. 33, pp. 21524–21538, 2020. [arXiv](#)
- [7] D. Eriksson et al., “Scalable global optimization via local Bayesian optimization,” in *Adv. Neural Inf. Process. Syst.*, vol. 32, 2019. [arXiv](#)
- [8] D. Eriksson and M. Jankowiak, “High-dimensional Bayesian optimization with sparse axis-aligned subspaces,” in *Proc. 37th Conf. Uncertainty Artif. Intell.*, 2021, pp. 493–503. [arXiv](#)
- [9] A. Brohan et al., “RT-1: Robotics Transformer for real-world control at scale,” *arXiv:2212.06817*, 2022. [arXiv](#)
- [10] A. Brohan et al., “RT-2: Vision-language-action models transfer Web knowledge to robotic control,” in *Proc. 7th Conf. Robot Learning*, 2023. [arXiv](#)
- [11] Open X-Embodiment Collaboration et al., “Open X-Embodiment: Robotic learning datasets and RT-X models,” in *Proc. IEEE Int. Conf. Robot. Autom.*, 2024. [arXiv](#)
- [12] M. J. Kim et al., “OpenVLA: An open-source vision-language-action model,” *arXiv:2406.09246*, 2024. [arXiv](#)
- [13] K. Black et al., “ π 0: A vision-language-action flow model for general robot control,” *arXiv:2410.24164*, 2024. [arXiv](#)
- [14] Physical Intelligence et al., “ π 0.5: A vision-language-action model with open-world generalization,” *arXiv:2504.16054*, 2025. [arXiv](#)
- [15] C. Zhang et al., “JoyAI-RA 0.1: A foundation model for robotic autonomy,” *arXiv:2604.20100*, 2026. [arXiv](#)
- [16] J. Yang et al., “SWE-agent: Agent-computer interfaces enable automated software engineering,” in *Adv. Neural Inf. Process. Syst.*, vol. 37, 2024. [arXiv](#)
- [17] A. Novikov et al., “AlphaEvolve: A Gemini-powered coding agent for designing advanced algorithms,” *arXiv:2506.13131*, 2025. [arXiv](#)
- [18] DeepSeek-AI, “DeepSeek-V3 technical report,” *arXiv:2412.19437*, 2024. [arXiv](#)
- [19] A. Yang et al., “Qwen2 technical report,” *arXiv:2407.10671*, 2024. [arXiv](#)
- [20] K. Bousmalis et al., “RoboCat: A self-improving generalist agent for robotic manipulation,” *Trans. Mach. Learn. Res.*, 2023. [arXiv](#)
- [21] O. M. Team et al., “Octo: An open-source generalist robot policy,” in *Proc. Robotics: Science and Systems*, 2024. [arXiv](#)
- [22] H. Walke et al., “BridgeData V2: A dataset for robot learning at scale,” in *Proc. Conf. Robot Learning*, 2023. [arXiv](#)
- [23] A. Khazatsky et al., “DROID: A large-scale in-the-wild robot manipulation dataset,” in *Proc. Robotics: Science and Systems*, 2024. [arXiv](#)
- [24] A. Nasiriany et al., “RoboCasa: Large-scale simulation of everyday tasks for generalist robots,” in *Proc. Robotics: Science and Systems*, 2024. [arXiv](#)
- [25] Y. Chen et al., “RoboTwin 2.0: A scalable data generator and benchmark with strong domain randomization for robust bimanual robotic manipulation,” *arXiv:2506.18088*, 2025. [arXiv](#)
- [26] AgiBot-World Contributors et al., “AgiBot World Colosseo: A large-scale manipulation platform for scalable and intelligent embodied systems,” *arXiv:2503.06669*, 2025. [arXiv](#)
- [27] C. Chi et al., “Diffusion Policy: Visuomotor policy learning via action diffusion,” *Int. J. Robot. Res.*, vol. 42, no. 13, pp. 1172–1198, 2023. [arXiv](#)
- [28] S. Yao et al., “ReAct: Synergizing reasoning and acting in language models,” in *Proc. Int. Conf. Learn. Representations*, 2023. [arXiv](#)
- [29] T. Schick et al., “Toolformer: Language models can teach themselves to use tools,” in *Adv. Neural Inf. Process. Syst.*, vol. 36, 2023. [arXiv](#)
- [30] N. Shinn et al., “Reflexion: Language agents with verbal reinforcement learning,” in *Adv. Neural Inf. Process. Syst.*, vol. 36, 2023. [arXiv](#)
- [31] D. R. Jones, M. Schonlau, and W. J. Welch, “Efficient global optimization of expensive black-box functions,” *J. Global Optim.*, vol. 13, no. 4, pp. 455–492, 1998, doi: 10.1023/A:1008306431147. [DOI](#)
- [32] J. Snoek, H. Larochelle, and R. P. Adams, “Practical Bayesian optimization of machine learning algorithms,” in *Adv. Neural Inf. Process. Syst.*, vol. 25, 2012. [arXiv](#)
- [33] B. Shahriari, K. Swersky, Z. Wang, R. P. Adams, and N. de Freitas, “Taking the human out of the loop: A review of Bayesian optimization,” *Proc. IEEE*, vol. 104, no. 1, pp. 148–175, Jan. 2016, doi: 10.1109/JPROC.2015.2494218. [arXiv](#)